

UNIT- III

UNIT – III

Thumb Instruction Set - Register Usage - Other Branch Instructions - Data Processing Instructions - Single-Register and Multi Register Load-Store Instructions - Stack - Software Interrupt Instructions

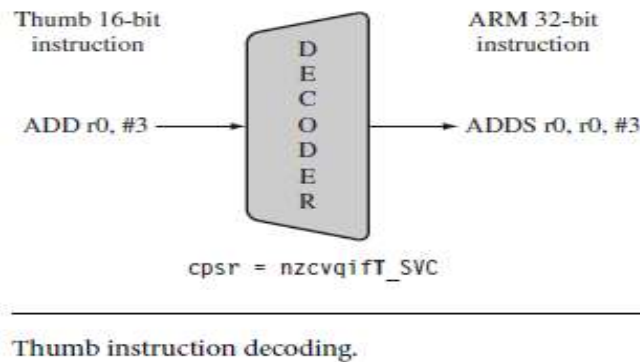
THUMB INSTRUCTION

- Thumb encodes a subset of the 32-bit ARM instructions into a 16-bit instruction set space.
- Thumb has higher performance than ARM on a processor with a 16-bit data bus, but lower performance than ARM on a 32-bit data bus.
- Thumb is used for memory-constrained systems.
- Thumb has higher *code density*—the space taken up in memory by an executable program—than ARM.
- For memory-constrained embedded systems, for example, mobile phones and PDAs, code density is very important.
- Cost pressures also limit memory size, width, and speed.
- On average, a Thumb implementation of the same code takes up around 30% less memory than the equivalent ARM implementation.

Example:

ARM code	Thumb code
ARMDivide ; IN: r0(value),r1(divisor) ; OUT: r2(MODulus),r3(DIVide)	ThumbDivide ; IN: r0(value),r1(divisor) ; OUT: r2(MODulus),r3(DIVide)
MOV r3,#0 loop SUBS r0,r0,r1 ADDGE r3,r3,#1 BGE loop ADD r2,r0,r1	MOV r3,#0 loop ADD r3,#1 SUB r0,r1 BGE loop SUB r3,#1 ADD r2,r0,r1
5 × 4 = 20 bytes	6 × 2 = 12 bytes

THUMB INSTRUCTION DECODING



THUMB INSTRUCTION SET

Thumb instruction set.

Mnemonics	THUMB ISA	Description
ADC	v1	add two 32-bit values and carry
ADD	v1	add two 32-bit values
AND	v1	logical bitwise AND of two 32-bit values
ASR	v1	arithmetic shift right
B	v1	branch relative
BIC	v1	logical bit clear (AND NOT) of two 32-bit values
BKPT	v2	breakpoint instructions
BL	v1	relative branch with link
BLX	v2	branch with link and exchange
BX	v1	branch with exchange
CMN	v1	compare negative two 32-bit values
CMP	v1	compare two 32-bit integers
EOR	v1	logical exclusive OR of two 32-bit values
LDM	v1	load multiple 32-bit words from memory to ARM registers
LDR	v1	load a single value from a virtual address in memory
LSL	v1	logical shift left
LSR	v1	logical shift right
MOV	v1	move a 32-bit value into a register
MUL	v1	multiply two 32-bit values
MVN	v1	move the logical NOT of 32-bit value into a register
NEG	v1	negate a 32-bit value
ORR	v1	logical bitwise OR of two 32-bit values
POP	v1	pops multiple registers from the stack
PUSH	v1	pushes multiple registers to the stack
ROR	v1	rotate right a 32-bit value
SBC	v1	subtract with carry a 32-bit value
STM	v1	store multiple 32-bit registers to memory
STR	v1	store register to a virtual address in memory
SUB	v1	subtract two 32-bit values
SWI	v1	software interrupt
TST	v1	test bits of a 32-bit value

THUMB REGISTER USAGE

- In Thumb state, only the low registers *r0* to *r7* are fully accessible.
- The higher registers *r8* to *r12* are only accessible with MOV, ADD, or CMP instructions.
- CMP and all the data processing instructions that operate on low registers update the condition flags in the *cpsr*.
- There are no MSR- and MRS-equivalent Thumb instructions.
- To alter the *cpsr* or *spsr*, switch into ARM state to use MSR and MRS.

Summary of Thumb register usage.

Registers	Access
<i>r0–r7</i>	fully accessible
<i>r8–r12</i>	only accessible by MOV, ADD, and CMP
<i>r13 sp</i>	limited accessibility
<i>r14 lr</i>	limited accessibility
<i>r15 pc</i>	limited accessibility
<i>cpsr</i>	only indirect access
<i>spsr</i>	no access

ARM-THUMB INTERWORKING

- *ARM-Thumb interworking* is the name given to the method of linking ARM and Thumb code together for both assembly and C/C++.
- It handles the transition between the two states.
- Extra code, called a *veneer*, is sometimes needed to carry out the transition.
- ATPCS defines the ARM and Thumb procedure call standards.
- To call a Thumb routine from an ARM routine, the core has to change state.
 - This state change is shown in the *T* bit of the *cpsr*.
- The BX and BLX branch instructions cause a switch between ARM and Thumb state while branching to a routine.
 - The BX *lr* instruction returns from a routine, also with a state switch if necessary.
 - The BLX instruction was introduced in ARMv5T.
- On ARMv4T cores the linker uses a *veneer* to switch state on a subroutine call.
- Instead of calling the routine directly, the linker calls the *veneer*, which switches to Thumb state using the BX instruction.
- There are two versions of the BX or BLX instructions: an ARM instruction and a Thumb equivalent.
 - The ARM BX instruction enters Thumb state only if bit 0 of the address in *Rn* is set to binary 1; otherwise it enters ARM state.
 - The Thumb BX instruction does the same.

BRANCH INSTRUCTIONS

Syntax: BX Rm
 BLX Rm | label

BX	Thumb version branch exchange	$pc = Rn \ \& \ 0xfffffffffe$ $T = Rn[0]$
BLX	Thumb version of the branch exchange with link	$lr = (\text{instruction address after the BLX}) + 1$ $pc = label, \ T = 0$ $pc = Rm \ \& \ 0xfffffffffe, \ T = Rm[0]$

BX

Branch with exchange (branch with possible state switch)

BX Rm

$pc = Rm \ \& \ 0xfffffffffe$ $T = Rm \ \& \ 1$

Examples

BX lr ; return from ARM or Thumb subroutine

BX r0 ; branch to ARM or Thumb function pointer r0

BLX

Branch with link and exchange (subroutine call with possible state switch)

BLX <address22>

$lr = ret + 1$; $pc = \text{<address22>}$ $T = 0$ (switch to ARM state)

Example

This example shows a small code fragment that uses both the ARM and Thumb versions of the BX instruction. The branch address into Thumb has the lowest bit set. This sets the *T* bit in the *cpsr* to Thumb state. The return address is not automatically preserved by the BX instruction. Rather the code sets the return address explicitly using a MOV instruction prior to the branch:

ARM code

CODE32 ; word aligned

LDR r0, =thumbCode+1 ; +1 to enter Thumb state

MOV lr, pc ; set the return address

BX r0 ; branch to Thumb code & mode

; continue here

Thumb code

CODE16 ; halfword aligned

thumbCode

ADD r1, #1

BX lr ; return to ARM code & state

- Replacing the BX instruction with BLX simplifies the calling of a Thumb routine since it sets the return address in the link register *lr*:

CODE32

LDR r0, =thumbRoutine+1 ; enter Thumb state

BLX r0 ; jump to Thumb code

; continue here

CODE16

thumbRoutine

ADD r1, #1

BX r14 ; return to ARM code and state

OTHER BRANCH INSTRUCTIONS

- There are two variations of the standard branch instruction, or B.
- The first is similar to the ARM version and is conditionally executed; the branch range is limited to a signed 8-bit immediate, or -256 to +254 bytes.
- The second version removes the conditional part of the instruction and expands the effective branch range to a signed 11-bit immediate, or -2048 to +2046 bytes.
- The conditional branch instruction is the only conditionally executed instruction in Thumb state.

Syntax: B<cond> label

B label

BL label

B	branch	$pc = label$
BL	branch with link	$pc = label$ $lr = (\text{instruction address after the BL}) + 1$

B

Branch relative

B<cond> <address8>

B <address11>

Branches to the given address or label. The address is stored as a relative offset.

Examples

B label ; branch unconditionally to a label

BGT loop ; conditionally continue a loop

BL

Relative branch with link (subroutine call)

BL <address22>

lr = ret+1; pc = <address22> None

- The BL instruction is not conditionally executed and has an approximate range of ± 4 MB.
- This range is possible because BL (and BLX) instructions are translated into a pair of 16-bit Thumb instructions.
- The first instruction in the pair holds the high part of the branch offset, and the second the low part. These instructions must be used as a pair.

Example:

The code here shows the various instructions used to return from a BL subroutine call:

MOV pc, lr

BX lr

POP {pc}

To return, set the *pc* to the value in *lr*.

DATA PROCESSING INSTRUCTIONS

- The data processing instructions manipulate data within registers.
- They include move instructions, arithmetic instructions, shifts, logical instructions, comparison instructions, and multiply instructions.
- The Thumb data processing instructions are a subset of the ARM data processing instructions.
- Most Thumb data processing instructions operate on low registers and update the *cpsr*.
- The exceptions are

MOV Rd,Rn

ADD Rd,Rm

CMP Rn,Rm

ADD sp, #immediate

SUB sp, #immediate

ADD Rd,sp,#immediate

ADD Rd,pc,#immediate

which can operate on the higher registers *r8–r14* and the *pc*.

ARITHMETIC INSTRUCTIONS

ADC	add two 32-bit values and carry	$Rd = Rd + Rm + C \text{ flag}$
ADD	add two 32-bit values	$Rd = Rn + immediate$ $Rd = Rd + immediate$ $Rd = Rd + Rm$ $Rd = Rd + Rm$ $Rd = (pc \& 0xfffffff) + (immediate \ll 2)$ $Rd = sp + (immediate \ll 2)$ $sp = sp + (immediate \ll 2)$
SBC	subtract with carry a 32-bit value	$Rd = Rd - Rm - \text{NOT}(C \text{ flag})$
SUB	subtract two 32-bit values	$Rd = Rn - immediate$ $Rd = Rd - immediate$ $Rd = Rn - Rm$ $sp = sp - (immediate \ll 2)$

ADD

Add two 32-bit values

(i) ADD Ld, Ln, #<immed3>

$Ld = Ln + \text{<immed3>}$

CPSR Updated

(ii) ADD Ld, #<immed8>

$Ld = Ld + \text{<immed8>}$

CPSR Updated

(iii) ADD Ld, Ln, Lm

$Ld = Ln + Lm$

CPSR Updated

(iv) ADD Ld, pc, #<immed8>*4

$Ld = pc + 4 * \text{<immed8>}$

(v) ADD Ld, sp, #<immed8>*4

$Ld = sp + 4 * \text{<immed8>}$

Examples

ADD r0, r1, #4 ; $r0 = r1 + 4$

ADDS pc, lr, #4 ; jump to lr+4, restoring the cpsr

ADC

Add two 32-bit values and carry

ADC Ld, Lm

$Ld = Ld + Lm + C$

CPSR Updated

Example:

PRE

cpsr = nzcvtIFT_SVC

r1 = 0x80000000

r2 = 0x10000000

ADD r0, r1, r2

POST

r0 = 0x90000000

cpsr = NzcvIFT_SVC

It takes two low registers *r1* and *r2* and adds them together. The result is then placed into register *r0*, overwriting the original contents. The *cpsr* is also updated.

SUB

Subtract two 32-bit values

(i) SUB Ld, Ln, #<immed3>

$Ld = Ln - \text{<immed3>}$ Updated

(ii) SUB Ld, #<immed8>

$Ld = Ld - \text{<immed8>}$ Updated

Examples

SUBS r0, r0, #1 ; r0-=1, setting flags

SUB r0, r1, r1, LSL #2 ; $r0 = -3 * r1$

SUBS pc, lr, #4 ; jump to lr-4, set cpsr=spsr

SBC

Subtract with carry

(i) SBC Ld, Lm

$Ld = Ld - Lm - (\sim C)$ Updated

(ii) SBC r1, r1, r3 ; subtract high words and borrow

MULTIPLY INSTRUCTIONS

MUL

Multiply

MUL	multiply two 32-bit values	$Rd = (Rm * Rd)[31:0]$
------------	----------------------------	------------------------

MUL Ld, Lm

Ld = Lm*Ld

CPSR Updated

LOGICAL INSTRUCTIONS

AND	logical bitwise AND of two 32-bit values	$Rd = Rd \& Rm$
ORR	logical bitwise OR of two 32-bit values	$Rd = Rd \text{ OR } Rm$
EOR	logical exclusive OR of two 32-bit values	$Rd = Rd \text{ EOR } Rm$
BIC	logical bit clear (AND NOT) of two 32-bit values	$Rd = Rd \text{ AND NOT}(Rm)$

AND

Logical bitwise AND of two 32-bit values

AND Ld, Lm

Ld = Ld & Lm

ORR

Logical bitwise OR of two 32-bit values

ORR Ld, Lm

Ld = Ld | Lm Updated

Example

ORR r0, r0, #1 << 13 ; set bit 13 of r0

EOR

Logical exclusive OR of two 32-bit values

EOR Ld, Lm

Ld = Ld ^ Lm Updated

BIC

Logical bit clear (AND NOT) of two 32-bit values

BIC Ld, Lm

Ld = Ld & ~Lm Updated

Examples

BIC r0, r0, #1 << 22 ; clear bit 22 of r0

COMPARISON INSTRUCTIONS

CMN	compare negative two 32-bit values	$Rn + Rm$ sets flags
CMP	compare two 32-bit integers	$Rn - \text{immediate}$ sets flags $Rn - Rm$ sets flags
TST	test bits of a 32-bit value	$Rn \text{ AND } Rm$ sets flags

CMN

Compare negative

CMN Ln, Lm

CPSR flags set on the result of (Ln + Lm)

Eg:

CMN r0, #3 ; compare r0 with -3

CMP

Compare two 32-bit integers

(i) CMP Ln, #<immed8>

CPSR flags set on the result of (Ln - <immed8>)

(ii) CMP Rn, Rm

CPSR flags set on the result of (Rn - Rm)

TST

Test bits of a 32-bit value

TST Ln, Lm

Set the cpsr on the result of (Ln & Lm)

MOVE INSTRUCTIONS

MOV	move a 32-bit value into a register	$Rd = \text{immediate}$ $Rd = Rn$ $Rd = Rm$
MVN	move the logical NOT of a 32-bit value into a register	$Rd = \text{NOT}(Rm)$

MOV

Move a 32-bit value into a register

(i) MOV Ld, #<immed8>

Ld = <immed8>

CPSR Updated

(ii) MOV Ld, Ln

Ld = Ln

CPSR Updated

MVN

Move the logical not of a 32-bit value into a register

MVN Ld, Lm

Ld = ~Lm

CPSR Updated

SHIFT AND ROTATE INSTRUCTIONS

Thumb deviates from the ARM style in that the barrel shift operations. In Thumb instruction set ASR, LSL, LSR, and ROR are separate instructions. These are equivalent instructions for MOV in ARM.

ASR

Arithmetic shift right for Thumb

(i) ASR Ld, Lm, #<immed5>

Ld = Lm ASR #<immed5>

CPSR Updated

(ii) ASR Ld, Ls

Ld = Ld ASR Ls[7:0]

CPSR Updated

LSL

Logical shift left for Thumb

(i) LSL Ld, Lm, #<immed5>

Ld = Lm LSL #<immed5>

CPSR Updated

(ii) LSL Ld, Ls THUMBv1

Ld = Ld LSL Ls[7:0]

CPSR Updated

LSR

Logical shift right for Thumb

(i) LSR Ld, Lm, #<immed5>

Ld = Lm LSR #<immed5>

CPSR Updated

(ii) LSR Ld, Ls

Ld = Ld LSR Ls[7:0]

CPSR Updated

ROR

Rotate right for Thumb

(i) ROR Ld, Ls

Ld = Ld ROR Ls[7:0]

CPSR Updated

Example:

Logical left shift (LSL) instruction

PRE

r2 = 0x00000002

r4 = 0x00000001

LSL r2, r4

POST

r2 = **0x00000004**

r4 = 0x00000001

SINGLE-REGISTER LOAD-STORE INSTRUCTIONS

The Thumb instruction set supports load and storing registers, or LDR and STR. These instructions use two preindexed addressing modes: offset by register and offset by immediate.

LDR	load word into a register	$Rd \leftarrow mem32[address]$
STR	save word from a register	$Rd \rightarrow mem32[address]$
LDRB	load byte into a register	$Rd \leftarrow mem8[address]$
STRB	save byte from a register	$Rd \rightarrow mem8[address]$
LDRH	load halfword into a register	$Rd \leftarrow mem16[address]$
STRH	save halfword into a register	$Rd \rightarrow mem16[address]$
LDRSB	load signed byte into a register	$Rd \leftarrow SignExtend(mem8[address])$
LDRSH	load signed halfword into a register	$Rd \leftarrow SignExtend(mem16[address])$

Addressing modes

Type	Syntax
Load/store register	$[Rn, Rm]$
Base register + offset	$[Rn, \#immediate]$
Relative	$[pc sp, \#immediate]$

- The offset by register uses a base register Rn plus the register offset Rm .
- The second uses the same base register Rn plus a 5-bit immediate or a value dependent on the data size.
- The 5-bit offset encoded in the instruction is multiplied by one for byte accesses, two for 16-bit accesses, and four for 32-bit accesses.

LDR

Load a single value from a virtual address in memory

(i) $LDR\{B|H\} Ld, [Ln, \#<immed5>*<size>]$

(ii) $LDR\{B|H|SB|SH\} Ld, [Ln, Lm]$

(iii) $LDR Ld, [pc, \#<immed8>*4]$

(iv) $LDR Ld, [sp, \#<immed8>*4]$

STR

Store a single value to a virtual address in memory

(i) $STR\{B|H\} Ld, [Ln, \#<immed5>*<size>]$

(ii) STR{|B|H} Ld, [Ln, Lm]

(iii) STR Ld, [sp, #<immed8>*4] THUMBv1

Example

This example shows two Thumb instructions that use a preindex addressing mode. Both use the same pre-condition.

PRE

mem32[0x90000] = 0x00000001

mem32[0x90004] = 0x00000002

mem32[0x90008] = 0x00000003

r0 = 0x00000000

r1 = 0x00090000

r4 = 0x00000004

LDR r0, [r1, r4] ; register

POST

r0 = 0x00000002

r1 = 0x00090000

r4 = 0x00000004

LDR r0, [r1, #0x4] ; immediate

POST

r0 = 0x00000002

Both instructions carry out the same operation. The only difference is the second LDR uses a fixed offset, whereas the first one depends on the value in register *r4*.

MULTIPLE-REGISTER LOAD-STORE INSTRUCTIONS

The Thumb versions of the load-store multiple instructions are reduced forms of the ARM load-store multiple instructions. They only support the increment after (IA) addressing mode.

Syntax : <LDM|STM>IA Rn!, {low Register list}

LDMIA	load multiple registers	$\{Rd\}^{*N} \leftarrow \text{mem32}[Rn + 4 * N], Rn = Rn + 4 * N$
STMIA	save multiple registers	$\{Rd\}^{*N} \rightarrow \text{mem32}[Rn + 4 * N], Rn = Rn + 4 * N$

- N is the number of registers in the list of registers.
- These instructions always update the base register Rn after execution.
- The base register and list of registers are limited to the low registers $r0$ to $r7$.

LDM

Load multiple 32-bit words from memory to ARM registers

LDMIA Rn!, <register_list>

STM

Store multiple 32-bit registers to memory

STMIA Rn!, <register_list>

STMIA r4!, {r0, r1} ; *r4=r0, *(r4+4)=r1, r4+=8

Example

This example saves registers $r1$ to $r3$ to memory addresses 0x9000 to 0x900c. It also updates base register $r4$. Note that the update character ! is not an option, unlike with the ARM instruction set.

PRE

$r1 = 0x00000001$

$r2 = 0x00000002$

$r3 = 0x00000003$

$r4 = 0x9000$

STMIA r4!, {r1, r2, r3}

POST

$\text{mem32}[0x9000] = 0x00000001$

mem32[0x9004] = 0x00000002

mem32[0x9008] = 0x00000003

r4 = 0x900c

STACK INSTRUCTIONS

The Thumb stack operations are different from the equivalent ARM instructions because they use the more traditional POP and PUSH concept. This is equivalent of LDMFD & STMFD instructions respectively of ARM.

Syntax: POP {low_register_list[, pc]}
 PUSH {low_register_list[, lr]}

POP	pop registers from the stacks	$Rd^{*N} \leftarrow mem32[sp + 4 * N], sp = sp + 4 * N$
PUSH	push registers on to the stack	$Rd^{*N} \rightarrow mem32[sp + 4 * N], sp = sp - 4 * N$

- Stack pointer is fixed as register *r13* in Thumb operations and *sp* is automatically updated.
- The list of registers is limited to the low registers *r0* to *r7*.
- The PUSH register list also can include the link register *lr*.
- The POP register list can include the *pc*.
- This provides support for subroutine entry and exit. The stack instructions only support full descending stack operations.

Example

In this example we use the POP and PUSH instructions. The subroutine Thumb Routine is called using a branch with link (BL) instruction.

; Call subroutine

BL ThumbRoutine

; continue

ThumbRoutine

PUSH {r1, lr} ; enter subroutine

MOV r0, #2

POP {r1, pc} ; return from subroutine

The link register *lr* is pushed onto the stack with register *r1*. Upon return, register *r1* is popped off the stack, as well as the return address being loaded into the *pc*. This returns from the subroutine.

SOFTWARE INTERRUPT INSTRUCTION

Thumb software interrupt (SWI) instruction causes a software interrupt exception. If any interrupt or exception flag is raised in Thumb state, the processor automatically reverts back to ARM state to handle the exception.

Syntax: SWI *immediate*

SWI	software interrupt	<i>lr_svc</i> = address of instruction following the SWI <i>spsr_svc</i> = <i>cpsr</i> <i>pc</i> = vectors + 0x8 <i>cpsr</i> mode = SVC <i>cpsr</i> I = 1 (mask IRQ interrupts) <i>cpsr</i> T = 0 (ARM state)
-----	--------------------	--

The Thumb SWI instruction has the same effect and nearly the same syntax as the ARM equivalent. It differs in that the SWI number is limited to the range 0 to 255 and it is not conditionally executed.

Example

This example shows the execution of a Thumb SWI instruction. The processor goes from Thumb state to ARM state after execution.

PRE

```
cpsr = nzcVqifT_USER  
pc = 0x00008000  
lr = 0x003fffff ; lr = r14  
r0 = 0x12
```

```
0x00008000 SWI 0x45
```

POST

```
cpsr = nzcVqIfT_SVC  
spsr = nzcVqifT_USER  
pc = 0x00000008  
lr = 0x00008002  
r0 = 0x12
```

2 MARKS

1. What is THUMB? (Nov 2017)

- Thumb encodes a subset of the 32-bit ARM instructions into a 16-bit instruction set space.
- Thumb has higher performance than ARM on a processor with a 16-bit data bus, but lower performance than ARM on a 32-bit data bus.

2. What are the special features of THUMB?

- Thumb is used for memory-constrained systems.
- Thumb has higher *code density*—the space taken up in memory by an executable program—than ARM.
- For memory-constrained embedded systems, for example, mobile phones and PDAs, code density is very important.
- Cost pressures also limit memory size, width, and speed.
- On average, a Thumb implementation of the same code takes up around 30% less memory than the equivalent ARM implementation.
- Code density was the main driving force for the Thumb instruction set

3. How to alter the PSR contents in Thumb state?

There is no direct access to the *cpsr* or *spsr*. To alter the *cpsr* or *spsr*, you must switch into ARM state to use MSR and MRS

4. Define ARM-Thumb interworking. (May 2016)

ARM-Thumb interworking is the name given to the method of linking ARM and Thumb code together for both assembly and C/C++. It handles the transition between the two states. Extra code, called a *veneer*, is sometimes needed to carry out the transition.

5. What is veneer? State the use of it.

Extra code, called a *veneer*, is sometimes needed to carry out the transition between ARM and Thumb state. On ARMv4T cores the linker uses a veneer to switch state on a subroutine call. Instead of calling the routine directly, the linker calls the veneer, which switches to Thumb state using the BX instruction.

6. How to switch from ARM state to Thumb state?

To call a Thumb routine from an ARM routine, the core has to change state. This state change is shown in the *T* bit of the *cpsr*. The BX and BLX branch instructions cause a switch between ARM and Thumb state while branching to a routine. The BX *lr* instruction returns from a routine, also with a state switch if necessary.

7. What is TST?

TST => test bits of a 32-bit value

Syntax: *Rn* AND *Rm*

sets flags

8. List the load and store instructions of Thumb.

The Thumb instruction set supports load and storing registers, or LDR and STR. These instructions use two preindexed addressing modes: offset by register and offset by immediate.

9. Give about stack instructions of Thumb (Nov 2017)

The Thumb stack operations are different from the equivalent ARM instructions because Thumb uses the POP and PUSH concept. there is no stack pointer in the instruction. This is because the stack pointer is fixed as register *r13* in Thumb operations and *sp* is automatically updated. The list of registers is limited to the low registers *r0* to *r7*. The PUSH register list also can include the link register *lr*; similarly the POP register list can include the *pc*.

10. Define SWI (May 2016) (May 2017)

- SWI means Software Interrupt
- SWI instruction causes a software interrupt exception. If any interrupt or exception flag is raised in Thumb state, the processor automatically reverts back to ARM state to handle the exception.

11. Give the instruction which give the negative of a number.

NEG Negate value in Thumb

NEG Ld, Lm

Ld = -Lm CPSR Updated

12. Give about REV instruction.

REV Reverse bytes within a word or halfword.

REV<cond> Rd, Rm

Rd[31:24] = Rm[07:00]; Rd[23:16] = Rm[15:08];

Rd[15:08] = Rm[23:16]; Rd[07:00] = Rm[31:24]

13. Give the instruction used to apply breakpoints in Thumb.

BKPT => Breakpoint instruction

BKPT <immed8>

The breakpoint instruction causes a prefetch data abort, unless overridden by debug hardware. This immediate can be used to hold debug information such as the breakpoint number.

14. Compare the stack instructions used in ARM and Thumb

In ARM, for stack operations LDMFD & STMFD are used where as the equivalent instruction in Thumb is POP & PUSH respectively.

15. List out the registers of Thumb with its access privileges (May 2016)

Registers	Access
<i>r0–r7</i>	fully accessible
<i>r8–r12</i>	only accessible by MOV, ADD, and CMP
<i>r13 sp</i>	limited accessibility
<i>r14 lr</i>	limited accessibility
<i>r15 pc</i>	limited accessibility
<i>cpsr</i>	only indirect access
<i>spsr</i>	no access

16. List out the Thumb instructions that can access higher registers.

- MOV Rd,Rn
- ADD Rd,Rm
- CMP Rn,Rm
- ADD sp, #immediate
- SUB sp, #immediate
- ADD Rd,sp,#immediate
- ADD Rd,pc,#immediate

These instructions can operate on the higher registers *r8–r14* and the *pc*.

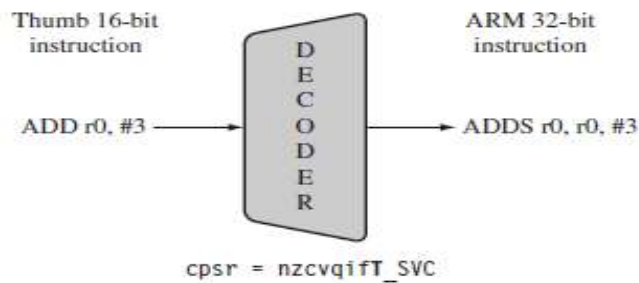
17. Is barrel shifter is used in Thumb instruction set?

Thumb deviates from the ARM style in that the barrel shift operations (ASR, LSL, LSR, and ROR) are separate instructions.

18. List out the instruction to access co-processor

There are no Thumb instructions to access the coprocessors

19. What is instruction decoding in Thumb?



Thumb instruction decoding.

20. What are conditional instructions available in Thumb?

Interworking uses the branch exchange (BX) instruction and branch exchange with link (BLX) instruction to change state and jump to a specific routine. In Thumb, only the branch instructions are conditionally executed.

21. What is Thumb instruction set? List out any two of it. (Nov 2016)

Each Thumb instruction is related to a 32-bit ARM instruction. A simple Thumb ADD instruction being decoded into an equivalent ARM ADD instruction.

Examples:

ADD – add two 32-bit values

ADC - add two 32-bit values and carry

22. Mention any two differences between single register and multi register (Nov 2016)

Single register loads a single variable from memory to register whereas Multi register loads multiple values from memory to register.

LDR – loads single word into a register

LDM – load multiple registers